

Webex Contact Center

New

Beta

This feature is currently in Beta. Contact your Cisco team if you want access to this feature.

In this model:

- A middleware application owns the external messaging channel integration.
- Webex Contact Center owns the contact-center task lifecycle.
- Administrators configure the messaging channel, asset, webhook target, entry point, and flow.
- The middleware application uses public APIs to create tasks and append inbound messages, receives outbound messages through the configured webhook, subscribes to task lifecycle events, and retrieves transcripts.

Responsibilities

anchor

Use this model when your middleware application is responsible for receiving inbound messages from the external messaging channel, forwarding them to Webex Contact Center, and delivering outbound messages from Webex Contact Center back to the end customer.

The middleware application is responsible for:

- Hosting the channel integration
- Obtaining and refreshing OAuth tokens
- Calling the Create Task API for the initial inbound message
- Calling the Task Messages API for subsequent inbound messages
- Processing outbound webhooks received from the asset-level webhook URL
- Subscribing to task lifecycle and inbound message subscription webhooks

Webex Contact Center is responsible for:

- Validating and routing Custom Messaging interactions
- Running the configured flow
- Presenting the conversation to agents
- Generating task lifecycle and task message webhook events
- Storing transcripts for later retrieval

Before You Begin

anchor

Before integrating, ensure that the administrator has configured the following in Control Hub:

- A Custom Messaging channel with:
 - Messaging policies for text and attachments
- A Custom Messaging asset with:
 - Business address
 - Webhook URL
 - Webhook secret
- A Custom Messaging entry point mapped to a Custom Messaging flow
- Other Contact Center routing objects required for the interaction, such as queues, teams, and multimedia profiles

Authentication For Custom Messaging Integrations

anchor

- For the middleware application to call the Tasks APIs, you must use a [Service App](#).
- For scope definitions, refer to the [Integrations guide](#).
- The general [Authentication guide](#) explains token acquisition and token refresh behavior.

Create And Authorize A Service App

1. Sign in to the developer portal and create a new Service App.
2. Capture the client secret when it is first shown. After the app is submitted for authorization, regenerate the secret if it was not saved.
3. Assign the required scopes:
 - cjp:task_write
 - cjp:task_read
4. Ask the administrator to authorize the Service App in Control Hub under **Apps**.
5. After authorization, return to the Service App in the developer portal, select the authorized organization, provide the client secret, and

generate the access token.

Use the resulting access token when calling the Create Task API and the Task Messages API.

For this integration pattern, ensure that:

- The Service App is authorized for the organization
- The access token includes `cjp:task_write` for Create Task and Task Messages
- The access token includes `cjp:task_read` when task read access is required

Refresh Tokens

For long-running access, retain the refresh token and use the standard OAuth refresh flow described in the [Authentication guide](#) before the current access token expires.

For server-to-server integrations, store the client secret, access token, and refresh token securely, and rotate or regenerate credentials according to your organization's security practices.

anchorMessaging Policy Behavior

anchor

Messaging policies are configured on the Custom Messaging channel in Control Hub under Contact Center > Channels & Assets > Channels > Custom Messaging. These policies control:

- Maximum message characters
- Whether attachments are allowed
- Maximum number of attachments
- Maximum total attachment size
- Maximum single attachment size
- Allowed file types

If an inbound message violates the configured channel policy, the entire message is rejected rather than partially accepted.

If administrators update the messaging policy, inbound middleware traffic starts using the updated policy almost immediately after the configuration change is applied. For outbound agent messaging, the updated policy is applied after the next relevant desktop reload or session refresh.

anchorSubscribe Before Sending Traffic

anchor

Before using the Create Task API and the Task Messages API, configure subscription webhooks for the task and task-message events that your middleware application needs to process.

At a minimum, subscribe to:

- `task:failed`
- `task-message:appended`
- `task-message:append-failed`

It is also recommended to subscribe to `task:new` and any other task lifecycle events your implementation needs.

Use the [Subscriptions API](#) to register the subscription, and refer to the [Webhooks guide](#) for payload details and correlation behavior.

anchorCreate The Initial Inbound Task

anchor

Use the [Create Task API](#) to create the initial Custom Messaging task.

For Custom Messaging, Create Task supports inbound task creation only. Outbound messages are sent later by flows or agents and are delivered to the middleware application through the configured webhook.

Example request:

```
{
  "origin": {
    "id": "customer@example.com",
    "name": "Customer Name"
  },
  "destination": {
    "id": "support@channel.biz",
    "type": "businessAddress"
  },
  "channelType": "customMessaging",
  "channel": "my-custom-channel",
  "globalVariables": {
    "customerTier": "gold"
  }
}
```

```
    },
    "channelParams": {
      "type": "text-with-attachments",
      "message": {
        "aliasId": "90a896bc-5eb5-4d15-8e11-f62e99751c21",
        "text": "Hello, I need help with my order. Please see the attached screenshot.",
        "attachments": [
          {
            "fileName": "order-screenshot.png",
            "mimeType": "image/png",
            "fileUrl": "https://cdn.partner.example.com/files/order-screenshot.png"
          }
        ]
      },
      "timestamp": 1779785508649
    }
  }
}
```

Important request fields:

Parameter	Description	Mandatory
origin.id	Middleware-managed customer identifier	Yes
destination.id	Business address configured on the asset	Yes
channelType	Must be customMessaging	Yes
channel	Configured Custom Messaging channel name	Yes
channelParams.type	Message type, currently text or text-with-attachments	Yes
channelParams.message.aliasId	Middleware message identifier	Yes
channelParams.message.text	Customer message text	Yes
channelParams.message.attachments	Attachment list for text-with-attachments	Conditional
channelParams.message.attachments[].fileName	Middleware-provided file name	Conditional
channelParams.message.attachments[].mimeType	Attachment media type	Conditional
channelParams.message.attachments[].fileUrl	HTTPS URL that the platform can retrieve	Conditional
channelParams.message.timestamp	Customer message timestamp in epoch milliseconds	Yes

Message type validation checks from the current routing model:

- text: Requires non-empty text. Attachments are not supported.
- text-with-attachments: Includes text and attachments. The attachments field must be present. Empty text is accepted when attachment content is present. Each attachment entry requires fileName, mimeType, and HTTPS fileUrl.

The guide shows one representative Custom Messaging example. For the full set of supported request combinations and validation rules, refer to the [Create Task API](#).

Example response:

```
{
  "meta": {
    "orgId": "cb5901fb-ab66-4377-a7bc-5f0b21896952"
  },
  "data": {
    "id": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7"
  }
}
```

Use data.id as the created task identifier. The middleware application should retain this value for subsequent Task Messages API calls and to correlate later task lifecycle subscription webhooks for the interaction.

A 201 Created response acknowledges that the Create Task request was accepted, but it does not guarantee that the task was fully created successfully. Use task:new as the success acknowledgement and task:failed as the failure acknowledgement.

anchorSubscribe To Task Lifecycle Events

anchor

Use the [Subscriptions API](#) for task lifecycle and inbound task-message webhook events.

Important event families for Custom Messaging integrations include:

- task:new
- task:connect
- task:connected
- task:ended
- task:failed
- task-message:appended
- task-message:append-failed

Use the [List Event Types API](#) to discover the latest supported event types and resource versions.

Use the [Webhooks guide](#) for detailed task and task-message event payload documentation.

Note that subscription-based `task-message:appended` and `task-message:append-failed` describe inbound task message flows. The outbound webhook documented in this guide is asset-configured and is not delivered through the Subscription API.

Representative inbound `task-message:appended` subscription webhook:

```
{
  "id": "4c2c4890-8a36-4dcc-9702-4c3a4aa1dc51",
  "specversion": "1.0",
  "type": "task-message:appended",
  "source": "/com/cisco/wxcc/ed9357d0-3f5d-4c6a-a4fd-111111111111",
  "comciscoorgid": "cb5901fb-ab66-4377-a7bc-5f0b21896952",
  "comciscotimestamp": 1779785508649,
  "datacontenttype": "application/json",
  "data": {
    "taskId": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7",
    "direction": "INBOUND",
    "messageDirection": "INBOUND",
    "channelType": "customMessaging",
    "channel": "my-custom-channel",
    "origin": "customer@example.com",
    "destination": "support@channel.biz",
    "createdTime": 1779785508649,
    "eventDetails": "Appended Message to chat successfully.",
    "channelParams": {
      "type": "text-with-attachments",
      "message": {
        "aliasId": "90a896bc-5eb5-4d15-8e11-f62e99751c21",
        "text": "I have more details to share about my order. Please see the attached invoice.",
        "timestamp": 1779785508649,
        "attachments": [
          {
            "fileName": "invoice.pdf",
            "mimeType": "application/pdf",
            "fileUrl": "https://cdn.partner.example.com/files/invoice.pdf"
          }
        ]
      }
    }
  },
  "reason": null,
  "errorMessage": null
}
```

Representative `task:failed` subscription webhook:

```
{
  "id": "8be34eb4-d64f-4552-96cc-c9b6e73063c2",
  "specversion": "1.0",
  "type": "task:failed",
  "source": "/com/cisco/wxcc/ed9357d0-3f5d-4c6a-a4fd-222222222222",
  "comciscoorgid": "cb5901fb-ab66-4377-a7bc-5f0b21896952",
  "comciscotimestamp": 1779785508649,
  "datacontenttype": "application/json",
  "data": {
    "taskId": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7",
    "origin": "customer@example.com",
    "destination": "support@channel.biz",
    "direction": "INBOUND",
    "channel": "my-custom-channel",
    "channelType": "customMessaging",
    "mediaMgr": "digitalmm",
    "reason": "CONVERSATION_ALREADY_OPEN",
    "errorMessage": "Conversation is already open with task id ab4ea619-a1ad-33a3-aed9-c2563a9bd6cc",
    "failureType": "failed",
    "createdTime": 1779785508649
  }
}
```

Common cases that can result in `task:failed` include:

- `CONVERSATION_ALREADY_OPEN`: a conversation is already active for the same customer and business address
- `CHANNEL_ASSET_UNDEFINED`: the destination business address does not resolve to a valid Custom Messaging asset
- `FEATURE_FLAG_DISABLED`: the organization is not enabled for Custom Messaging
- `ENTRY_POINT_NOT_FOUND`: the asset does not resolve to an active entry point
- `ORG_DIGITAL_CONTACT_LIMIT_EXCEEDED`: the organization has reached its active digital contact limit
- `CONVERSATION_CREATION_FAILED`: downstream conversation creation failed after the request was accepted
- Validation or policy failures, such as unsupported content, unsupported file types, attachment count limits, or attachment size limits

Representative `task-message:append-failed` subscription webhook:

```
{
  "id": "64ab4f4a-5c93-4339-b325-3878f289a88f",
  "specversion": "1.0",
  "type": "task-message:append-failed",
  "source": "/com/cisco/wxcc/ed9357d0-3f5d-4c6a-a4fd-333333333333",
  "comciscoorgid": "cb5901fb-ab66-4377-a7bc-5f0b21896952",
  "comciscotimestamp": 1779785508649,
  "datacontenttype": "application/json",
}
```

```

"data": {
  "taskId": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7",
  "direction": "INBOUND",
  "messageDirection": "INBOUND",
  "channelType": "customMessaging",
  "channel": "my-custom-channel",
  "origin": "customer@example.com",
  "destination": "support@channel.biz",
  "createdTime": 1779785508649,
  "eventDetails": null,
  "channelParams": null,
  "reason": "INVALID_CONTENT",
  "errorMessage": "The maximum number of attachments allowed is 1."
}
}

```

Use the following fields to correlate inbound message webhook processing:

- `data.taskId`: task identifier
- `data.channelParams.message.aliasId`: middleware message identifier for `task-message:appended` (when `channelParams` is present)
- `data.messageDirection`: direction of the appended message
- `id`: webhook event identifier

anchorAppend Subsequent Inbound Messages

anchor

Use the [Task Messages API](#) to append subsequent inbound customer messages to the same task.

Example request:

```

{
  "mediaType": "customMessaging",
  "channelParams": {
    "type": "text-with-attachments",
    "message": {
      "aliasId": "90a896bc-5eb5-4d15-8e11-f62e99751c21",
      "text": "I have more details to share. Please see the attached invoice.",
      "attachments": [
        {
          "fileName": "invoice.pdf",
          "mimeType": "application/pdf",
          "fileUrl": "https://cdn.partner.example.com/files/invoice.pdf"
        }
      ]
    },
    "timestamp": 1779785508649
  }
}

```

Example response:

```

{
  "meta": {
    "orgId": "cb5901fb-ab66-4377-a7bc-5f0b21896952"
  },
  "data": {
    "id": "90a896bc-5eb5-4d15-8e11-f62e99751c21"
  }
}

```

Use `data.id` as the append event correlation identifier. If request `aliasId` is a valid UUID, the platform reuses that value as the event identifier.

After a successful append, Webex Contact Center publishes the inbound `task-message:appended` subscription webhook for the same message, and the detailed event payload is documented in [Webhook Event Details](#).

Important request fields:

Parameter	Description	Mandatory
<code>mediaType</code>	Must be <code>customMessaging</code>	Yes
<code>channelParams.type</code>	Message payload type, such as <code>text</code> or <code>text-with-attachments</code>	Yes
<code>channelParams.message.aliasId</code>	Middleware message identifier for correlation	Yes
<code>channelParams.message.text</code>	Customer message text	Yes
<code>channelParams.message.attachments</code>	Attachment list used with <code>text-with-attachments</code>	Conditional
<code>channelParams.message.attachments[].fileName</code>	Middleware-provided file name	Conditional
<code>channelParams.message.attachments[].mimeType</code>	Attachment media type	Conditional
<code>channelParams.message.attachments[].fileUrl</code>	HTTPS URL that the platform can retrieve	Conditional
<code>channelParams.message.timestamp</code>	Customer message timestamp in epoch milliseconds	Yes

Common inbound append failure categories surfaced through `task-message:append-failed` are shown below, and the detailed event payload is documented in [Webhook Event Details](#):

Failure reason	Typical meaning	Typical next step
INVALID_CONTENT	The request payload violates message policy or validation rules, such as unsupported content, unsupported file type, too many attachments, or attachment size limits.	Correct the request payload and retry.
NOT_FOUND	The referenced task context could not be found.	Verify the task identifier before retrying.
INTERNAL_ERROR	The platform could not complete the append because of an unexpected internal failure.	Investigate the related failure event and retry.

anchorReceive Outbound Partner Webhooks

anchor

After the task is created, the associated flow can send outbound messages to the customer. After the task is connected to an agent, the agent can also send outbound messages. All outbound Custom Messaging messages are delivered to the middleware application through the asset-level webhook URL configured on the Control Hub asset.

Representative system-sent (flow-sent) outbound task-message:appended webhook:

```
{
  "id": "01de1010-a89c-4f3f-9dc1-9497164d3268",
  "specversion": "1.0",
  "type": "task-message:appended",
  "source": "/com/cisco/wxcc/15858e1d-5059-45fd-ae1-fc314c42a790",
  "comciscoorgid": "cb5901fb-ab66-4377-a7bc-5f0b21896952",
  "comciscotimestamp": 1779785486332,
  "datacontenttype": "application/json",
  "data": {
    "taskId": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7",
    "origin": "customer@example.com",
    "destination": "support@channel.biz",
    "messageDirection": "OUTBOUND",
    "senderType": "system",
    "channelType": "customMessaging",
    "channel": "my-custom-channel",
    "channelParams": {
      "type": "text",
      "message": {
        "aliasId": "4690d7d9-b1a7-4e5b-9568-ca867a26eb49",
        "text": "Thanks for contacting support. Please share your order number so we can help further.",
        "timestamp": 1779760284456,
        "attachments": []
      }
    }
  },
  "createdTime": 1779760284456
}
```

Representative agent-sent outbound task-message:appended webhook:

```
{
  "id": "11de1010-a89c-4f3f-9dc1-9497164d3268",
  "specversion": "1.0",
  "type": "task-message:appended",
  "source": "/com/cisco/wxcc/15858e1d-5059-45fd-ae1-fc314c42a790",
  "comciscoorgid": "cb5901fb-ab66-4377-a7bc-5f0b21896952",
  "comciscotimestamp": 1779785586332,
  "datacontenttype": "application/json",
  "data": {
    "taskId": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7",
    "origin": "customer@example.com",
    "destination": "support@channel.biz",
    "messageDirection": "OUTBOUND",
    "senderType": "agent",
    "senderId": "5f2ecb3f-1a83-41a9-9b70-c3f8b53b25a0",
    "channelType": "customMessaging",
    "channel": "my-custom-channel",
    "channelParams": {
      "type": "text",
      "message": {
        "aliasId": "5690d7d9-b1a7-4e5b-9568-ca867a26eb49",
        "text": "Hello, I am reviewing your request now. I will update you in a moment.",
        "timestamp": 1779785586332,
        "attachments": []
      }
    }
  },
  "createdTime": 1779785586332
}
```

Representative agent-sent outbound task-message:appended webhook with attachments:

```
{
  "id": "13cfff3d-8f88-4a49-90b9-9d49410abc54",
  "specversion": "1.0",
  "type": "task-message:appended",
```

```
"source": "/com/cisco/wxcc/15858e1d-5059-45fd-ae1-fc314c42a790",
"comciscoorgid": "cb5901fb-ab66-4377-a7bc-5f0b21896952",
"comciscotimestamp": 1779786678879,
"datacontenttype": "application/json",
"data": {
  "taskId": "044ea619-a0ad-41a3-9ed9-c2563a9bd6b7",
  "origin": "customer@example.com",
  "destination": "support@channel.biz",
  "messageDirection": "OUTBOUND",
  "senderType": "agent",
  "senderId": "5f2ecb3f-1a83-41a9-9b70-c3f8b53b25a0",
  "channelType": "customMessaging",
  "channel": "my-custom-channel",
  "channelParams": {
    "type": "text-with-attachments",
    "message": {
      "aliasId": "e5474344-4cab-4f9b-920e-1890117f9c10",
      "text": "I am sharing the requested document.",
      "timestamp": 1779786678831,
      "attachments": [
        {
          "url": "https://attachment-api.example.com/api/v1/attachments/87da5468-9e09-49db-883e-dce1e8d62fac?token=<signed-url-token>",
          "mimeType": "application/pdf",
          "fileName": "order-details.pdf"
        }
      ]
    }
  },
  "createdTime": 1779786678831
}
```

Important webhook fields:

Field	Description
id	Webhook event identifier
data.taskId	Task identifier for the interaction
data.messageDirection	OUTBOUND for outbound delivery to the middleware application
data.senderType	system for flow/system messages or agent for agent-sent messages
data.senderId	Present for agent-sent outbound messages
data.channelParams.type	Outbound message payload type such as text or text-with-attachments
data.channelParams.message.aliasId	Message identifier
data.channelParams.message.attachments	Present when the outbound message includes files

For generic webhook signature, timestamp, replay-protection, and version-header handling, refer to [Using Webhooks](#).

For BYOC outbound task-message:appended, the webhook version header is:

```
X-WebexCC-Webhook-Version: task-message:1.0.0
```

Outbound attachment URLs depend on the organization's attachment-encryption configuration:

- When attachment encryption is enabled, outbound attachment URLs point to encrypted attachment content
- Otherwise, URLs pointing to plain attachments are returned

When attachment encryption is enabled, the middleware application must use the Webex Decryption SDK to decrypt those attachments before further processing. For the decryption workflow and SDK steps, refer to the [Decrypting Attachments](#) section in the [Guide for Digital Transcripts](#).

If outbound delivery fails internally, there is no equivalent outbound webhook failure event unless future implementation changes introduce one.

anchorAttachment URL Requirements

anchor

These requirements apply to middleware-provided attachment URLs used by:

- Create Task
- Task Messages
- Flow/system-sent Custom Messaging attachments where the platform must retrieve the file

Requirements:

- The attachment URL must use https
- The attachment URL must be retrievable by the platform
- File type is resolved from the URL extension when present
- If the URL has no usable extension, the platform uses the provided filename as the fallback
- If the effective file type is unsupported, the message is rejected
- File size must be determinable from remote metadata such as Content-Length
- If file size cannot be determined reliably, the message is rejected

These rules are separate from agent desktop attachment uploads, which follow a different UI-driven validation path.

anchorSecurity, PCI, And Malware Handling

anchor

Custom Messaging webhooks use the same signature, timestamp, replay-protection, and version-header validation model as other Webex Contact Center webhooks. For the generic validation steps, see [Using Webhooks](#).

For message content handling:

- PCI-sensitive text is masked and processing continues
- For inbound create-task and inbound task-message flows, attachments that fail PCI or malware scanning are dropped while the message continues
- Inbound success payloads include eventDetails when some attachments were dropped
- Outbound middleware webhooks contain only the surviving attachments and do not explicitly list the dropped attachments
- Agent outbound attachment uploads are blocked before send when the file violates the enforced data security policy

Representative inbound success payload with masked PCI text:

```
{
  "data": {
    "taskId": "5bf807cb-8f35-4f40-8261-1e2d05d41123",
    "direction": "INBOUND",
    "messageDirection": "INBOUND",
    "channelType": "customMessaging",
    "channel": "my-custom-channel",
    "origin": "customer@example.com",
    "destination": "support@channel.biz",
    "createdTime": 1780295173809,
    "eventDetails": "Chat Created Successfully",
    "channelParams": {
      "type": "text-with-attachments",
      "message": {
        "aliasId": "cd0857f0-6b07-4e57-b821-9ae04b7f8bc8",
        "text": "Hey need help with XXXX XXXX XXXX XXXX",
        "attachments": [],
        "timestamp": 1780295173809
      }
    },
    "reason": null,
    "errorMessage": null
  }
}
```

Representative inbound success payload when PCI or malware scanning drops some attachments:

```
{
  "data": {
    "taskId": "5bf807cb-8f35-4f40-8261-1e2d05d41123",
    "direction": "INBOUND",
    "messageDirection": "INBOUND",
    "channelType": "customMessaging",
    "channel": "my-custom-channel",
    "origin": "customer@example.com",
    "destination": "support@channel.biz",
    "createdTime": 1780300207453,
    "eventDetails": "Appended Message to chat successfully. PCI or malware scan failed for some attachments. Affected attachments were dropped.",
    "channelParams": {
      "type": "text-with-attachments",
      "message": {
        "aliasId": "4a5a6865-feb6-49c3-a966-d493eccd0cf1",
        "text": "My card XXXX XXXX XXXX XXXX",
        "attachments": [
          {
            "fileName": "basicText.txt",
            "mimeType": "text/plain",
            "fileUrl": "https://cdn.partner.example.com/files/basicText.txt"
          }
        ]
      },
      "timestamp": 1780300207453
    }
  }
}
```

anchorError Handling And Recovery

anchor

The middleware application should treat task:failed and task-message:append-failed as the main failure events for create-task and inbound append operations.

Important guidance:

- `task:failed` is the failure event for create-task failures
- `task-message:append-failed` is the failure event for inbound task-message failures
- Representative append-failure reasons include `INVALID_CONTENT`, `INTERNAL_ERROR`, and `NOT_FOUND`

For stuck-contact recovery:

- If the middleware application receives `task:failed` for a `CONVERSATION_ALREADY_OPEN` condition
- Take the earlier task identifier from the error message
- Call [End Task](#) only when you have confirmed that the earlier task should be closed.
- Be cautious before ending the earlier task, because it may still be in a valid queued or active state. Only use End Task when you have confirmed that the earlier task is stale and should be closed
- Retry Create Task only after the existing task has been cleaned up, if your workflow requires a retry

anchorFlow Support

anchor

Custom Messaging flows support:

- Send Custom Message
- Receive Message

Receive Message allows the flow to wait for an inbound customer message before the flow continues.

Flow debug and runtime outputs include fields such as:

- `ErrorCode`
- `Description`
- `ErrorMessage`
- `TrackingId`

These values are useful when investigating send or receive failures.

anchorRetrieve Transcripts

anchor

Use the [Captures API](#) to retrieve post-interaction transcripts for Custom Messaging tasks.

Custom Messaging transcripts are retrieved after transcript generation, subject to retention and capture availability.